

Machine Learning

Support Vector Machine

FAST

DISCOVERING
THE FUTURE

Topics of previous lectures

- ✓ Ingredients of Machine Learning
- ✓ Classification Basics
- ✓ Basic Linear Classifier
- ✓ K-Nearest Neighbours Classifier
- ✓ Naive Bayes Classifier
- ✓ Decision Trees

Topics of today's lecture

- Convex Optimization
- Support Vector Machine (SVM)
- Hard-margin SVM
- Soft-margin SVM
- Kernel methods

Background for Constrained Optimization

Consider the following optimization problem

$$\min_{x,y} f(x,y)$$

$$\text{subject to } g(x,y) = c$$

Background for Constrained Optimization

Consider the following optimization problem

$$\min_{x,y} f(x,y)$$

$$\text{subject to } g(x,y) = c$$

We can solve this task with the method of **Lagrange multipliers**:

Background for Constrained Optimization

Consider the following optimization problem

$$\min_{x,y} f(x,y)$$

$$\text{subject to } g(x,y) = c$$

We can solve this task with the method of **Lagrange multipliers**:

- form the Lagrange function (Lagrangian):

$$\Lambda(x,y,\lambda) = f(x,y) - \lambda(g(x,y) - c)$$

where $\lambda \in \mathbb{R}$ is the Lagrange multiplier

Background for Constrained Optimization

Consider the following optimization problem

$$\min_{x,y} f(x,y)$$

$$\text{subject to } g(x,y) = c$$

We can solve this task with the method of **Lagrange multipliers**:

- form the Lagrange function (Lagrangian):

$$\Lambda(x,y,\lambda) = f(x,y) - \lambda(g(x,y) - c)$$

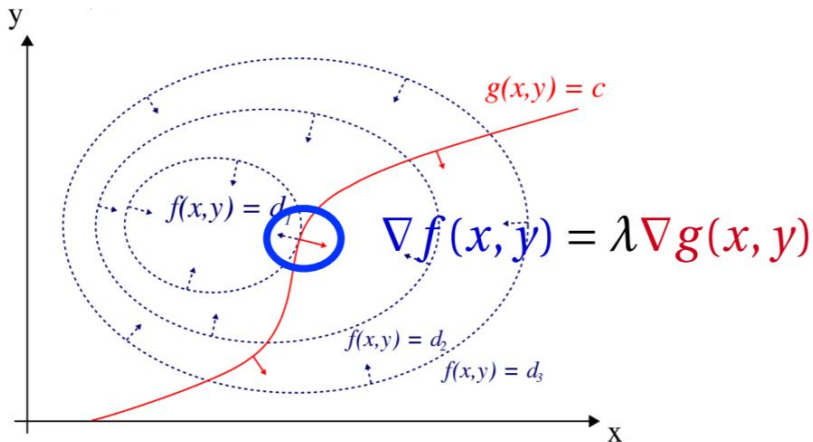
where $\lambda \in \mathbb{R}$ is the Lagrange multiplier

- Solve the unconstrained problem: $\nabla \Lambda(x,y,\lambda) = 0$, which is equivalent to

$$\nabla_{x,y} \Lambda(x,y,\lambda) = \nabla f(x,y) - \lambda \nabla g(x,y) = 0$$

$$\nabla_{\lambda} \Lambda(x,y,\lambda) = g(x,y) - c = 0$$

Background for Constrained Optimization



Background for Constrained Optimization

The following optimization problem

$$\min_{\mathbf{x}} f(\mathbf{x})$$

$$\text{subject to } g_i(\mathbf{x}) \leq 0, \quad \forall i = 1, \dots, m$$

is known as the **primal problem**, corresponding to *primal variables* $\mathbf{x}$.

Background for Constrained Optimization

The following optimization problem

$$\min_{\mathbf{x}} f(\mathbf{x})$$

$$\text{subject to } g_i(\mathbf{x}) \leq 0, \quad \forall i = 1, \dots, m$$

is known as the **primal problem**, corresponding to *primal variables* $\mathbf{x}$.
The associated Lagrangian **dual problem** is given by

$$\max_{\boldsymbol{\lambda} \in \mathbb{R}^m} \min_{\mathbf{x} \in \mathbb{R}^d} \Lambda(\mathbf{x}, \boldsymbol{\lambda})$$

$$\text{subject to } \boldsymbol{\lambda} \geq 0$$

where $\boldsymbol{\lambda}$ are the *dual variables*.

Background for Convex Optimization

- When $f(\cdot)$ is a convex function and the constraints form a convex set in the primal problem, then we will call it a **convex optimization** problem.

Background for Convex Optimization

- When $f(\cdot)$ is a convex function and the constraints form a convex set in the primal problem, then we will call it a **convex optimization** problem.
- In this case, the optimal solution of the dual problem is the same as the optimal solution of the primal problem (Karush–Kuhn–Tucker (KKT) theorem).

Background for Convex Optimization

- When $f(\cdot)$ is a convex function and the constraints form a convex set in the primal problem, then we will call it a **convex optimization** problem.
- In this case, the optimal solution of the dual problem is the same as the optimal solution of the primal problem (Karush–Kuhn–Tucker (KKT) theorem).

Definition (Convex set)

A set C is a convex set if $\forall x, y \in C$ and $\forall \theta \in [0, 1]$, we have

$$\theta x + (1 - \theta)y \in C$$

Background for Convex Optimization

- When $f(\cdot)$ is a convex function and the constraints form a convex set in the primal problem, then we will call it a **convex optimization** problem.
- In this case, the optimal solution of the dual problem is the same as the optimal solution of the primal problem (Karush–Kuhn–Tucker (KKT) theorem).

Definition (Convex set)

A set C is a convex set if $\forall x, y \in C$ and $\forall \theta \in [0, 1]$, we have

$$\theta x + (1 - \theta)y \in C$$

Definition (Convex function)

Let $f : X \rightarrow \mathbb{R}$ be a function such that X is a convex set, then f is a convex (concave) function if $\forall \mathbf{x}_1, \mathbf{x}_2 \in X$ and $\forall \theta \in [0, 1]$, we have

$$f(\theta \mathbf{x}_1 + (1 - \theta) \mathbf{x}_2) \leq \theta f(\mathbf{x}_1) + (1 - \theta) f(\mathbf{x}_2)$$

Theorem 1

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable, then $f(\mathbf{x})$ is convex iff $\forall \mathbf{x}_1, \mathbf{x}_2$

$$f(\mathbf{x}_2) \geq f(\mathbf{x}_1) + \nabla_{\mathbf{x}} f(\mathbf{x}_1)^T (\mathbf{x}_2 - \mathbf{x}_1).$$

Background for Convex Optimization

Theorem 1

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable, then $f(\mathbf{x})$ is convex iff $\forall \mathbf{x}_1, \mathbf{x}_2$

$$f(\mathbf{x}_2) \geq f(\mathbf{x}_1) + \nabla_{\mathbf{x}} f(\mathbf{x}_1)^T (\mathbf{x}_2 - \mathbf{x}_1).$$

Theorem 2

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is twice differentiable, then $f(\mathbf{x})$ is convex iff the Hessian matrix $\nabla_{\mathbf{x}}^2 f(\mathbf{x})$ is positive semidefinite.

Background for Convex Optimization

Theorem 1

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable, then $f(\mathbf{x})$ is convex iff $\forall \mathbf{x}_1, \mathbf{x}_2$

$$f(\mathbf{x}_2) \geq f(\mathbf{x}_1) + \nabla_{\mathbf{x}} f(\mathbf{x}_1)^T (\mathbf{x}_2 - \mathbf{x}_1).$$

Theorem 2

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is twice differentiable, then $f(\mathbf{x})$ is convex iff the Hessian matrix $\nabla_{\mathbf{x}}^2 f(\mathbf{x})$ is positive semidefinite.

Now let's look at two well-known classes of convex optimization problems.

Consider the case when the objective function is linear

$$\min_{\mathbf{x} \in \mathbb{R}^d} \mathbf{c}^T \mathbf{x}$$

subject to $\mathbf{A}\mathbf{x} \leq \mathbf{b}$,

where $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{c} \in \mathbb{R}^d$ and $\mathbf{b} \in \mathbb{R}^m$

Consider the case when the objective function is linear

$$\min_{\mathbf{x} \in \mathbb{R}^d} \mathbf{c}^T \mathbf{x}$$

subject to $\mathbf{A}\mathbf{x} \leq \mathbf{b}$,

where $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{c} \in \mathbb{R}^d$ and $\mathbf{b} \in \mathbb{R}^m$

The Lagrangian will be

$$\Lambda(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{A}\mathbf{x} - \mathbf{b}) = (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})^T \mathbf{x} - \boldsymbol{\lambda}^T \mathbf{b},$$

where $\boldsymbol{\lambda} \in \mathbb{R}^m$ is the vector of non-negative Lagrange multipliers.

Linear Programming

Consider the case when the objective function is linear

$$\min_{\mathbf{x} \in \mathbb{R}^d} \mathbf{c}^T \mathbf{x}$$

subject to $\mathbf{A}\mathbf{x} \leq \mathbf{b}$,

where $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{c} \in \mathbb{R}^d$ and $\mathbf{b} \in \mathbb{R}^m$

The Lagrangian will be

$$\Lambda(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{A}\mathbf{x} - \mathbf{b}) = (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})^T \mathbf{x} - \boldsymbol{\lambda}^T \mathbf{b},$$

where $\boldsymbol{\lambda} \in \mathbb{R}^m$ is the vector of non-negative Lagrange multipliers.

$$\nabla_{\mathbf{x}} \Lambda(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda} = 0$$

Linear Programming

Consider the case when the objective function is linear

$$\min_{\mathbf{x} \in \mathbb{R}^d} \mathbf{c}^T \mathbf{x}$$

subject to $\mathbf{A}\mathbf{x} \leq \mathbf{b}$,

where $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{c} \in \mathbb{R}^d$ and $\mathbf{b} \in \mathbb{R}^m$

The Lagrangian will be

$$\Lambda(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{A}\mathbf{x} - \mathbf{b}) = (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})^T \mathbf{x} - \boldsymbol{\lambda}^T \mathbf{b},$$

where $\boldsymbol{\lambda} \in \mathbb{R}^m$ is the vector of non-negative Lagrange multipliers.

$$\nabla_{\mathbf{x}} \Lambda(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda} = 0$$

The dual Lagrangian problem will be

$$\max_{\boldsymbol{\lambda} \in \mathbb{R}^m} -\boldsymbol{\lambda}^T \mathbf{b}$$

subject to $\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda} = 0, \quad \boldsymbol{\lambda} \geq 0$

Quadratic Programming

Consider the case when the objective function is quadratic

$$\min_{\mathbf{x} \in \mathbb{R}^d} \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x}$$

subject to $\mathbf{A} \mathbf{x} \leq \mathbf{b}$,

where $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{c} \in \mathbb{R}^d$, $\mathbf{b} \in \mathbb{R}^m$ and $\mathbf{Q} \in \mathbb{R}^{d \times d}$ is positive definite.

Quadratic Programming

Consider the case when the objective function is quadratic

$$\min_{\mathbf{x} \in \mathbb{R}^d} \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x}$$

subject to $\mathbf{A} \mathbf{x} \leq \mathbf{b}$,

where $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{c} \in \mathbb{R}^d$, $\mathbf{b} \in \mathbb{R}^m$ and $\mathbf{Q} \in \mathbb{R}^{d \times d}$ is positive definite.

The Lagrangian will be

$$\Lambda(\mathbf{x}, \boldsymbol{\lambda}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{A} \mathbf{x} - \mathbf{b}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})^T \mathbf{x} - \boldsymbol{\lambda}^T \mathbf{b},$$

Quadratic Programming

Consider the case when the objective function is quadratic

$$\min_{\mathbf{x} \in \mathbb{R}^d} \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x}$$

subject to $\mathbf{A} \mathbf{x} \leq \mathbf{b}$,

where $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{c} \in \mathbb{R}^d$, $\mathbf{b} \in \mathbb{R}^m$ and $\mathbf{Q} \in \mathbb{R}^{d \times d}$ is positive definite.
The Lagrangian will be

$$\Lambda(\mathbf{x}, \boldsymbol{\lambda}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{A} \mathbf{x} - \mathbf{b}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})^T \mathbf{x} - \boldsymbol{\lambda}^T \mathbf{b},$$

$$\nabla_{\mathbf{x}} \Lambda(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{Q} \mathbf{x} + (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda}) = 0$$

$$\mathbf{x} = -\mathbf{Q}^{-1}(\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})$$

Quadratic Programming

Consider the case when the objective function is quadratic

$$\min_{\mathbf{x} \in \mathbb{R}^d} \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x}$$

subject to $\mathbf{A} \mathbf{x} \leq \mathbf{b}$,

where $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{c} \in \mathbb{R}^d$, $\mathbf{b} \in \mathbb{R}^m$ and $\mathbf{Q} \in \mathbb{R}^{d \times d}$ is positive definite.
The Lagrangian will be

$$\Lambda(\mathbf{x}, \boldsymbol{\lambda}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{A} \mathbf{x} - \mathbf{b}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})^T \mathbf{x} - \boldsymbol{\lambda}^T \mathbf{b},$$

$$\nabla_{\mathbf{x}} \Lambda(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{Q} \mathbf{x} + (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda}) = 0$$

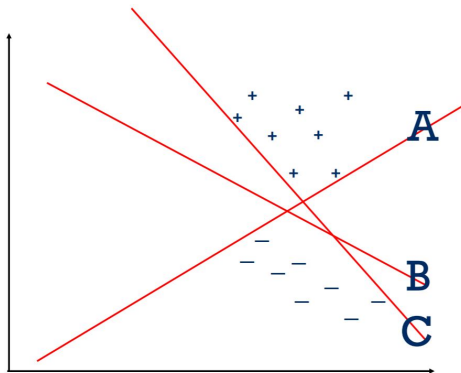
$$\mathbf{x} = -\mathbf{Q}^{-1}(\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})$$

The dual Lagrangian problem will be

$$\max_{\boldsymbol{\lambda} \in \mathbb{R}^m} -\frac{1}{2} (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})^T \mathbf{Q}^{-1} (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda}) - \boldsymbol{\lambda}^T \mathbf{b}$$

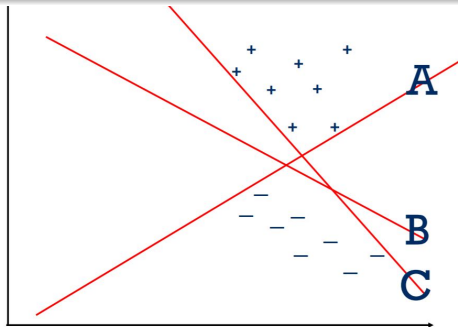
subject to $\boldsymbol{\lambda} \geq 0$

Motivation for SVM



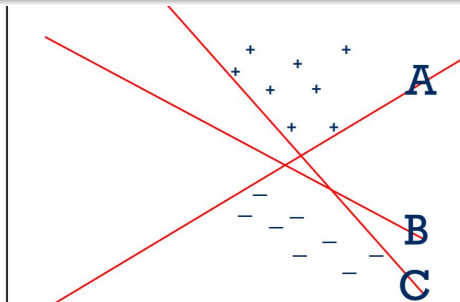
Motivation for SVM

There are infinitely many separating classifiers (decision boundaries) between linearly separable classes. Which one is the best in your opinion?



Motivation for SVM

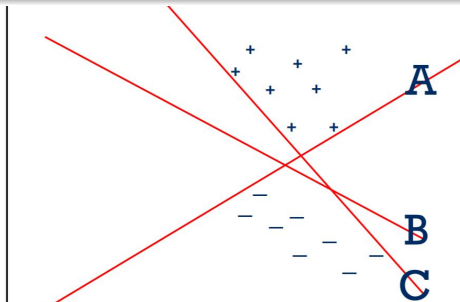
There are infinitely many separating classifiers (decision boundaries) between linearly separable classes. Which one is the best in your opinion?



On training data each is as good as any other

Motivation for SVM

There are infinitely many separating classifiers (decision boundaries) between linearly separable classes. Which one is the best in your opinion?



On training data each is as good as any other

Support Vector Machine learns the separating line B.

Support Vector Machine (SVM)

- Again a linear classification method

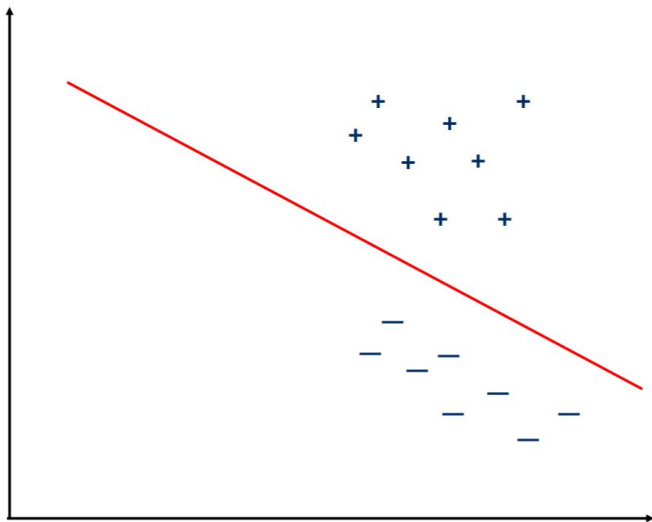
Support Vector Machine (SVM)

- Again a linear classification method
- If the classes are linearly separable then SVM finds a separating model

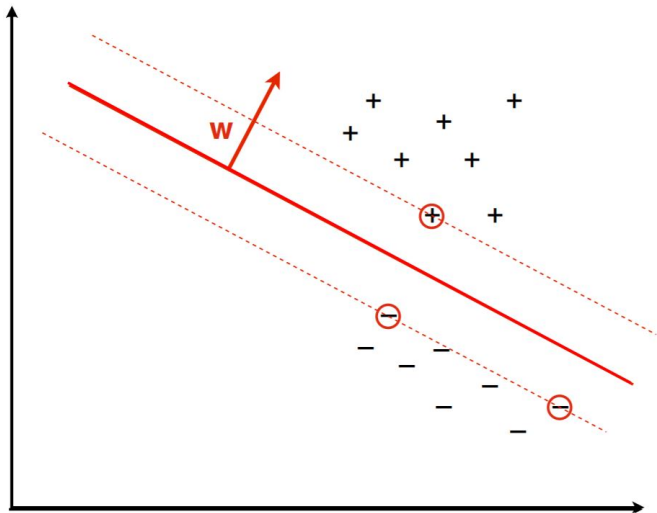
Support Vector Machine (SVM)

- Again a linear classification method
- If the classes are linearly separable then SVM finds a separating model
- SVM chooses the linear separating model which has the highest **margin** - distance between the decision boundary and the closest instance

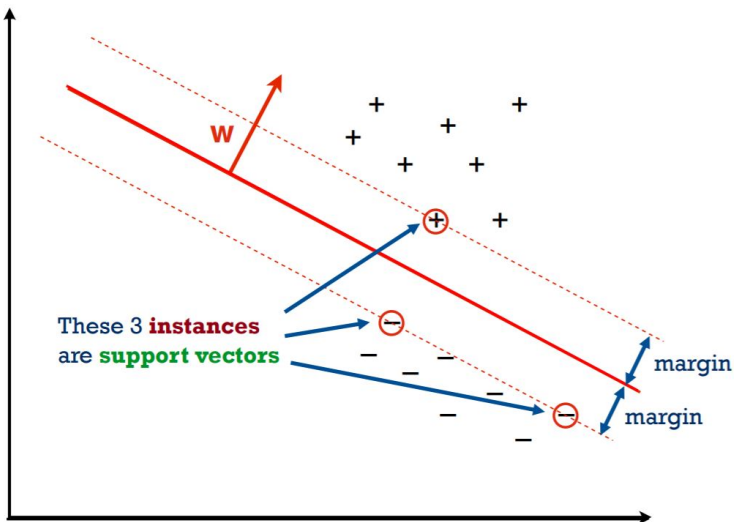
Support Vector Machine (SVM)



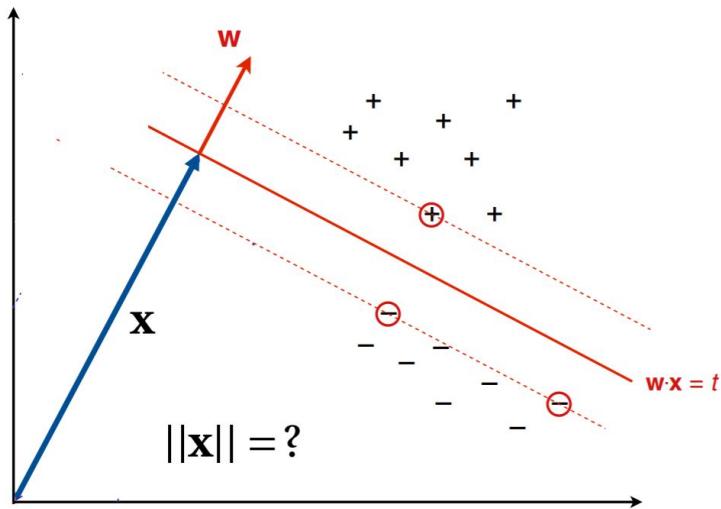
Support Vector Machine (SVM)



Support Vector Machine (SVM)

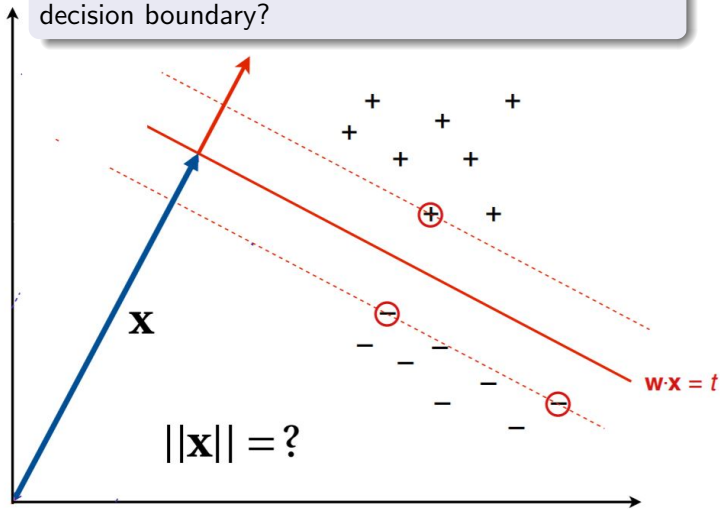


Calculating the margin



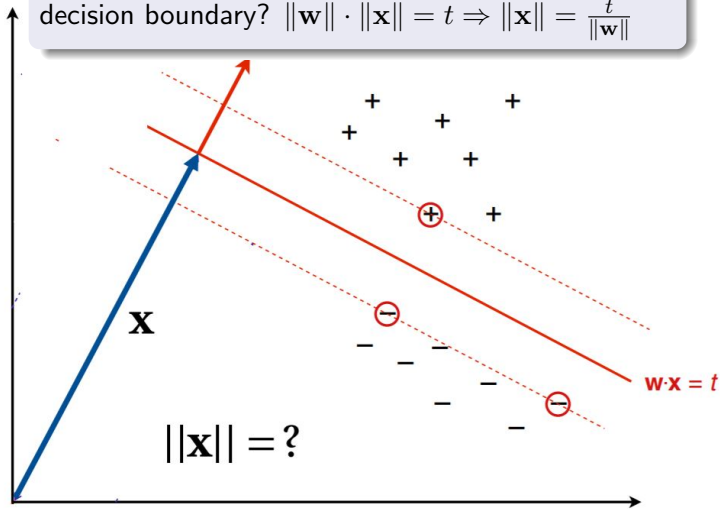
Calculating the margin

What is the distance between origin and the decision boundary?

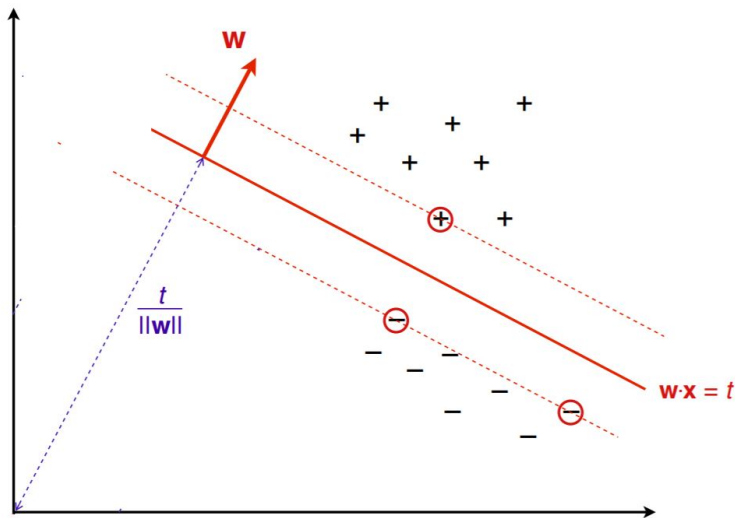


Calculating the margin

What is the distance between origin and the decision boundary? $\|\mathbf{w}\| \cdot \|\mathbf{x}\| = t \Rightarrow \|\mathbf{x}\| = \frac{t}{\|\mathbf{w}\|}$

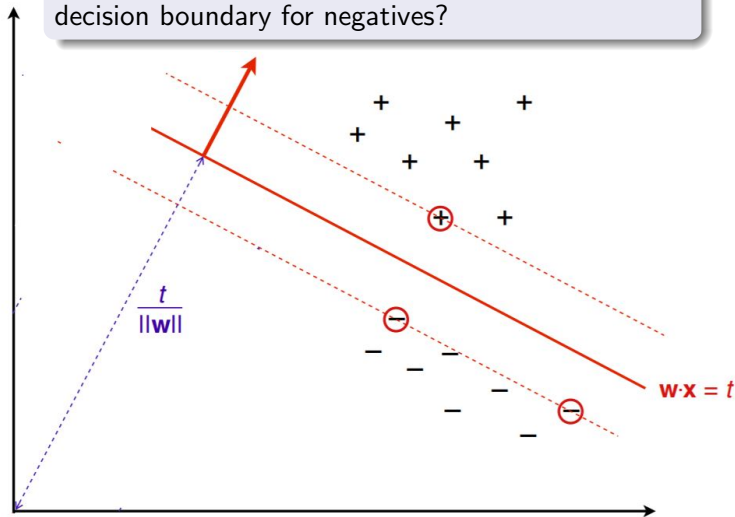


Calculating the margin



Calculating the margin

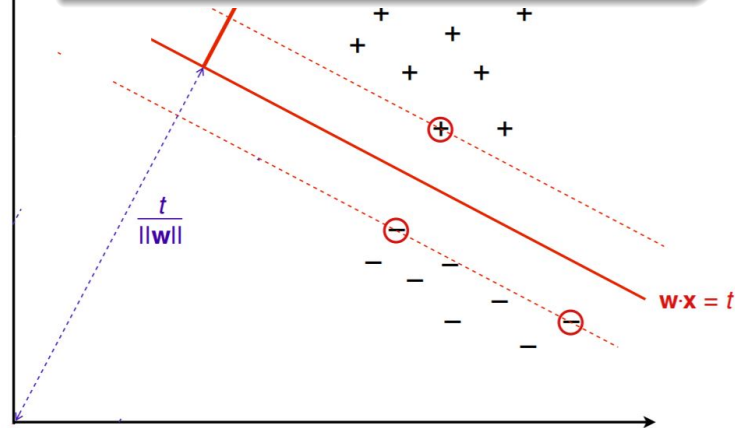
What is the distance between the origin and the decision boundary for negatives?



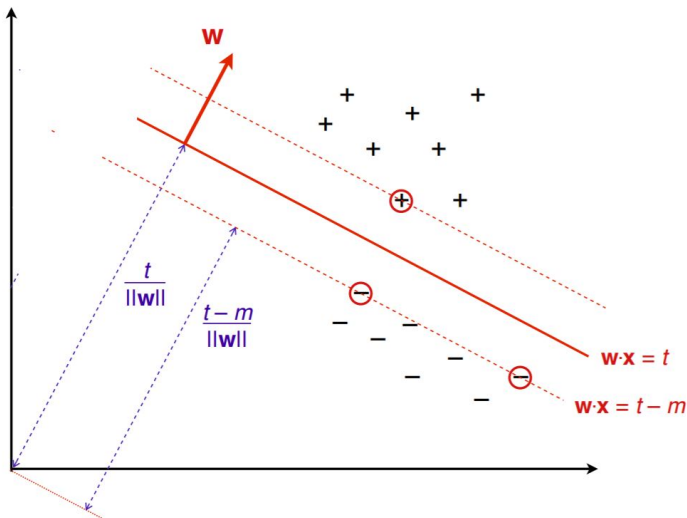
Calculating the margin

What is the distance between the origin and the decision boundary for negatives?

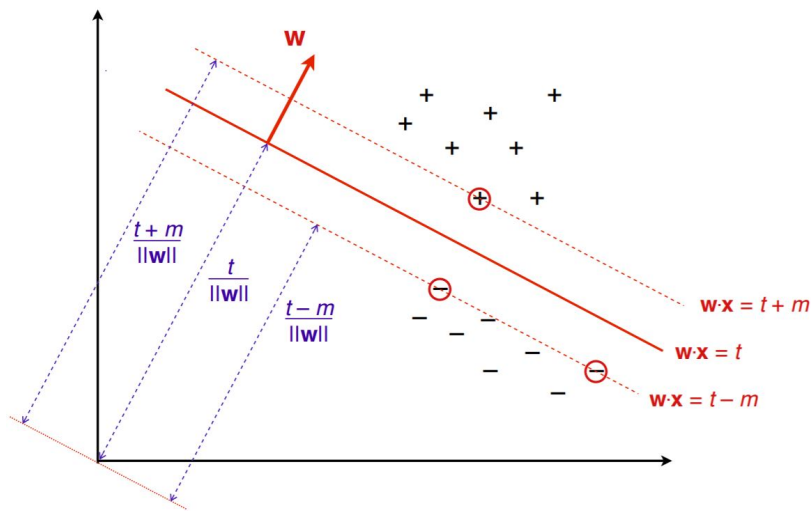
$$\|\mathbf{w}\| \cdot \|\mathbf{x}\| = t - m \Rightarrow \|\mathbf{x}\| = \frac{t-m}{\|\mathbf{w}\|}$$



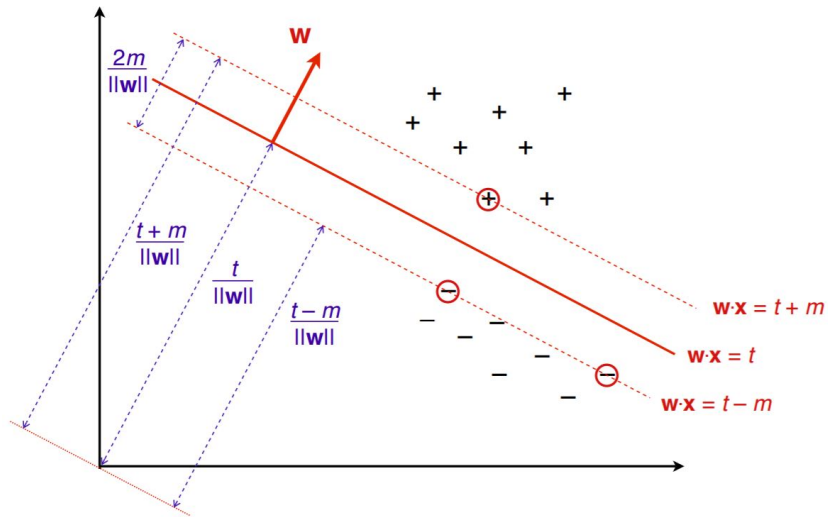
Calculating the margin



Calculating the margin

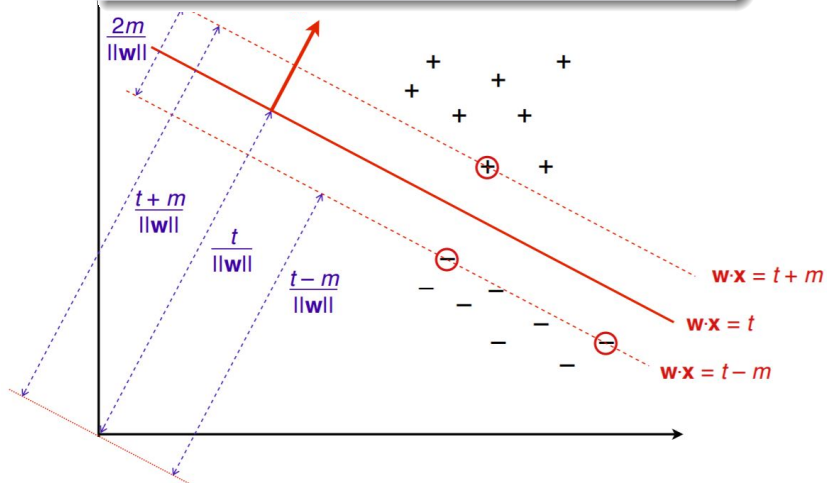


Key distances in SVM



Key distances in SVM

Since we are free to rescale t , w and m , it is customary to choose $m = 1$.



Optimization task in SVM

- Maximize the margin $\frac{1}{\|\mathbf{w}\|}$ such that
 - positives are at least by margin above the decision boundary:
 $\mathbf{w} \cdot \mathbf{x}_i \geq t + 1$
 - negatives are at least by margin below the decision boundary:
 $\mathbf{w} \cdot \mathbf{x}_i \leq t - 1$
- More conveniently and equivalently:

$$\mathbf{w}^*, t^* = \operatorname{argmin}_{\mathbf{w}, t} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1, \quad y_i \in \{-1, 1\}, \quad 1 \leq i \leq n$

Optimization task in SVM

- Maximize the margin $\frac{1}{\|\mathbf{w}\|}$ such that
 - positives are at least by margin above the decision boundary:
 $\mathbf{w} \cdot \mathbf{x}_i \geq t + 1$
 - negatives are at least by margin below the decision boundary:
 $\mathbf{w} \cdot \mathbf{x}_i \leq t - 1$
- More conveniently and equivalently:

$$\mathbf{w}^*, t^* = \operatorname{argmin}_{\mathbf{w}, t} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1, \quad y_i \in \{-1, 1\}, \quad 1 \leq i \leq n$

Suppose $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) = 1$. What can we say about $\mathbf{x}_i$?

$$\mathbf{w}^*, t^* = \underset{\mathbf{w}, t}{\operatorname{argmin}} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1, \quad y_i \in \{-1, 1\}, \quad 1 \leq i \leq n$

To solve this optimization problem, first let's form the Lagrange function

$$\Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) =$$

Hard-margin SVM

$$\mathbf{w}^*, t^* = \operatorname{argmin}_{\mathbf{w}, t} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1, \quad y_i \in \{-1, 1\}, \quad 1 \leq i \leq n$

To solve this optimization problem, first let's form the Lagrange function

$$\Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i (y_i(\mathbf{w} \cdot \mathbf{x}_i - t) - 1) =$$

Hard-margin SVM

$$\mathbf{w}^*, t^* = \operatorname{argmin}_{\mathbf{w}, t} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1, \quad y_i \in \{-1, 1\}, \quad 1 \leq i \leq n$

To solve this optimization problem, first let's form the Lagrange function

$$\begin{aligned} \Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i (y_i(\mathbf{w} \cdot \mathbf{x}_i - t) - 1) = \\ &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i y_i (\mathbf{w} \cdot \mathbf{x}_i) + \sum_{i=1}^n \alpha_i y_i t + \sum_{i=1}^n \alpha_i = \end{aligned}$$

Hard-margin SVM

$$\mathbf{w}^*, t^* = \operatorname{argmin}_{\mathbf{w}, t} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1, \quad y_i \in \{-1, 1\}, \quad 1 \leq i \leq n$

To solve this optimization problem, first let's form the Lagrange function

$$\begin{aligned} \Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i (y_i(\mathbf{w} \cdot \mathbf{x}_i - t) - 1) = \\ &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i y_i (\mathbf{w} \cdot \mathbf{x}_i) + \sum_{i=1}^n \alpha_i y_i t + \sum_{i=1}^n \alpha_i = \\ &= \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \mathbf{w} \cdot \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) + t \left(\sum_{i=1}^n \alpha_i y_i \right) + \sum_{i=1}^n \alpha_i \end{aligned}$$

Gradients of the Lagrangian

$$\Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \mathbf{w} \cdot \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) + t \left(\sum_{i=1}^n \alpha_i y_i \right) + \sum_{i=1}^n \alpha_i$$

Gradients of the Lagrangian

$$\Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \mathbf{w} \cdot \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) + t \left(\sum_{i=1}^n \alpha_i y_i \right) + \sum_{i=1}^n \alpha_i$$

$$\frac{\partial}{\partial t} \Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \sum_{i=1}^n \alpha_i y_i = 0$$

Gradients of the Lagrangian

$$\Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \mathbf{w} \cdot \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) + t \left(\sum_{i=1}^n \alpha_i y_i \right) + \sum_{i=1}^n \alpha_i$$

$$\frac{\partial}{\partial t} \Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \sum_{i=1}^n \alpha_i y_i = 0$$

$$\frac{\partial}{\partial \mathbf{w}} \Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0$$

Gradients of the Lagrangian

$$\Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \mathbf{w} \cdot \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) + t \left(\sum_{i=1}^n \alpha_i y_i \right) + \sum_{i=1}^n \alpha_i$$

$$\frac{\partial}{\partial t} \Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \sum_{i=1}^n \alpha_i y_i = 0$$

$$\frac{\partial}{\partial \mathbf{w}} \Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0$$

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

Plugging back into the Lagrangian

$$\Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \mathbf{w} \cdot \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) + t \left(\sum_{i=1}^n \alpha_i y_i \right) + \sum_{i=1}^n \alpha_i$$
$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad \sum_{i=1}^n \alpha_i y_i = 0$$

Plugging back into the Lagrangian

$$\Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \mathbf{w} \cdot \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) + t \left(\sum_{i=1}^n \alpha_i y_i \right) + \sum_{i=1}^n \alpha_i$$

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad \sum_{i=1}^n \alpha_i y_i = 0$$

$$\Lambda(\alpha_1, \dots, \alpha_n) = -\frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) \cdot \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) + \sum_{i=1}^n \alpha_i =$$

Plugging back into the Lagrangian

$$\Lambda(\mathbf{w}, t, \alpha_1, \dots, \alpha_n) = \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \mathbf{w} \cdot \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) + t \left(\sum_{i=1}^n \alpha_i y_i \right) + \sum_{i=1}^n \alpha_i$$

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad \sum_{i=1}^n \alpha_i y_i = 0$$

$$\begin{aligned} \Lambda(\alpha_1, \dots, \alpha_n) &= -\frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) \cdot \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right) + \sum_{i=1}^n \alpha_i = \\ &= -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j + \sum_{i=1}^n \alpha_i \end{aligned}$$

Applying KKT method on the Lagrangian

$$\Lambda(\alpha_1, \dots, \alpha_n) = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j + \sum_{i=1}^n \alpha_i$$

$$\alpha_1^*, \dots, \alpha_n^* = \operatorname{argmax}_{\alpha_1, \dots, \alpha_n} -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j + \sum_{i=1}^n \alpha_i$$

$$\text{subject to } \alpha_i \geq 0, \quad i = 1, \dots, n \text{ and } \sum_{i=1}^n \alpha_i y_i = 0$$

This task can be solved by quadratic optimisation solvers as we will see during the lab session.

Summary of SVM optimization

$$\mathbf{w}^*, t^* = \operatorname{argmin}_{\mathbf{w}, t} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1, \quad y_i \in \{-1, 1\}, \quad i = 1, \dots, n$

Summary of SVM optimization

$$\mathbf{w}^*, t^* = \operatorname{argmin}_{\mathbf{w}, t} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1, \quad y_i \in \{-1, 1\}, \quad i = 1, \dots, n$

We can solve the equivalent dual problem

$$\alpha_1^*, \dots, \alpha_n^* = \operatorname{argmax}_{\alpha_1, \dots, \alpha_n} -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j + \sum_{i=1}^n \alpha_i$$

subject to $\alpha_i \geq 0, \quad i = 1, \dots, n$ and $\sum_{i=1}^n \alpha_i y_i = 0$

Summary of SVM optimization

$$\mathbf{w}^*, t^* = \operatorname{argmin}_{\mathbf{w}, t} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1, \quad y_i \in \{-1, 1\}, \quad i = 1, \dots, n$

We can solve the equivalent dual problem

$$\alpha_1^*, \dots, \alpha_n^* = \operatorname{argmax}_{\alpha_1, \dots, \alpha_n} -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j + \sum_{i=1}^n \alpha_i$$

subject to $\alpha_i \geq 0, \quad i = 1, \dots, n$ and $\sum_{i=1}^n \alpha_i y_i = 0$

From the result we can calculate:

$$\mathbf{w}^* = \sum_{i=1}^n \alpha_i^* y_i \mathbf{x}_i \quad t^* = \mathbf{w}^* \cdot \mathbf{x}_i - y_i,$$

where $\mathbf{x}_i$ is a support vector and α_i is its weight.

Hard margin and soft margin SVM

- This was a **hard margin SVM** – we assumed that the classes are linearly separable

Hard margin and soft margin SVM

- This was a **hard margin SVM** – we assumed that the classes are linearly separable
- **Soft margin SVM** can tolerate **margin errors**: cases where an instance is inside the margin or even at the wrong side of the decision boundary

Hard margin and soft margin SVM

- This was a **hard margin SVM** – we assumed that the classes are linearly separable
- **Soft margin SVM** can tolerate **margin errors**: cases where an instance is inside the margin or even at the wrong side of the decision boundary
- The idea is to introduce slack variables $\xi_i \geq 0$, one for each instance, measuring the amount of margin error (or equal to 0 if no error)

- The task is the following:

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$

- The task is the following:

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$

- Here C is a regularisation parameter:

- The task is the following:

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$

- Here C is a regularisation parameter:
 - Higher C means more penalty on margin errors

- The task is the following:

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$

- Here C is a regularisation parameter:
 - Higher C means more penalty on margin errors
 - Lower C means less penalty on margin errors

- The task is the following:

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$

- Here C is a regularisation parameter:
 - Higher C means more penalty on margin errors
 - Lower C means less penalty on margin errors
- Higher C usually results in more support vectors, hence C is a **complexity parameter**

Lagrangian function

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$

$$\Lambda(\mathbf{w}, t, \xi_i, \alpha_i, \beta_i) =$$

Lagrangian function

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$$

$$\begin{aligned} \Lambda(\mathbf{w}, t, \xi_i, \alpha_i, \beta_i) = & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i - \\ & - \sum_{i=1}^n \alpha_i (y_i(\mathbf{w} \cdot \mathbf{x}_i - t) - (1 - \xi_i)) - \sum_{i=1}^n \beta_i \xi_i = \end{aligned}$$

Lagrangian function

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$$

$$\begin{aligned} \Lambda(\mathbf{w}, t, \xi_i, \alpha_i, \beta_i) &= \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i - \\ &\quad - \sum_{i=1}^n \alpha_i (y_i(\mathbf{w} \cdot \mathbf{x}_i - t) - (1 - \xi_i)) - \sum_{i=1}^n \beta_i \xi_i = \\ &= \Lambda(\mathbf{w}, t, \alpha_i) + \sum_{i=1}^n (C - \alpha_i - \beta_i) \xi_i \end{aligned}$$

Lagrangian function

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$$

$$\begin{aligned} \Lambda(\mathbf{w}, t, \xi_i, \alpha_i, \beta_i) &= \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i - \\ &\quad - \sum_{i=1}^n \alpha_i (y_i(\mathbf{w} \cdot \mathbf{x}_i - t) - (1 - \xi_i)) - \sum_{i=1}^n \beta_i \xi_i = \\ &= \Lambda(\mathbf{w}, t, \alpha_i) + \sum_{i=1}^n (C - \alpha_i - \beta_i) \xi_i \\ \frac{\partial}{\partial \xi_i} \Lambda(\mathbf{w}, t, \xi_i, \alpha_i, \beta_i) &= \end{aligned}$$

Lagrangian function

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$$

$$\begin{aligned} \Lambda(\mathbf{w}, t, \xi_i, \alpha_i, \beta_i) &= \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i - \\ &\quad - \sum_{i=1}^n \alpha_i (y_i(\mathbf{w} \cdot \mathbf{x}_i - t) - (1 - \xi_i)) - \sum_{i=1}^n \beta_i \xi_i = \\ &= \Lambda(\mathbf{w}, t, \alpha_i) + \sum_{i=1}^n (C - \alpha_i - \beta_i) \xi_i \end{aligned}$$

$$\frac{\partial}{\partial \xi_i} \Lambda(\mathbf{w}, t, \xi_i, \alpha_i, \beta_i) = C - \alpha_i - \beta_i = 0$$

Lagrangian function

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$$

$$\begin{aligned} \Lambda(\mathbf{w}, t, \xi_i, \alpha_i, \beta_i) &= \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i - \\ &\quad - \sum_{i=1}^n \alpha_i (y_i(\mathbf{w} \cdot \mathbf{x}_i - t) - (1 - \xi_i)) - \sum_{i=1}^n \beta_i \xi_i = \\ &= \Lambda(\mathbf{w}, t, \alpha_i) + \sum_{i=1}^n (C - \alpha_i - \beta_i) \xi_i \end{aligned}$$

$$\frac{\partial}{\partial \xi_i} \Lambda(\mathbf{w}, t, \xi_i, \alpha_i, \beta_i) = C - \alpha_i - \beta_i = 0 \Rightarrow \alpha_i \leq C,$$

since $\beta_i \geq 0$

Summary of soft-margin SVM

$$\mathbf{w}^*, t^*, \xi_i^* = \operatorname{argmin}_{\mathbf{w}, t, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$

Summary of soft-margin SVM

$$\mathbf{w}^*, t^*, \xi_i^* = \underset{\mathbf{w}, t, \xi_i}{\operatorname{argmin}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$$

We can solve the equivalent dual problem

$$\alpha_1^*, \dots, \alpha_n^* = \underset{\alpha_1, \dots, \alpha_n}{\operatorname{argmax}} -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j + \sum_{i=1}^n \alpha_i$$

$$\text{subject to } 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \text{ and } \sum_{i=1}^n \alpha_i y_i = 0$$

Summary of soft-margin SVM

$$\mathbf{w}^*, t^*, \xi_i^* = \underset{\mathbf{w}, t, \xi_i}{\operatorname{argmin}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$$

We can solve the equivalent dual problem

$$\alpha_1^*, \dots, \alpha_n^* = \underset{\alpha_1, \dots, \alpha_n}{\operatorname{argmax}} -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j + \sum_{i=1}^n \alpha_i$$

$$\text{subject to } 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \text{ and } \sum_{i=1}^n \alpha_i y_i = 0$$

From the result we can calculate:

$$\mathbf{w}^* = \sum_{i=1}^n \alpha_i^* y_i \mathbf{x}_i \quad t^* = \mathbf{w}^* \cdot \mathbf{x}_i - y_i,$$

where $\mathbf{x}_i$ is a support vector and α_i is its weight.

Pro: usually **generalizes** better than the other discussed linear methods due to margin maximization

Soft-margin SVM

- Pro:** usually **generalizes** better than the other discussed linear methods due to margin maximization
- Pro:** can be used with kernels (discussed soon)

Soft-margin SVM

- Pro:** usually **generalizes** better than the other discussed linear methods due to margin maximization
- Pro:** can be used with kernels (discussed soon)
- Pro:** works even if the classes are not linearly separable

Soft-margin SVM

- Pro:** usually **generalizes** better than the other discussed linear methods due to margin maximization
- Pro:** can be used with kernels (discussed soon)
- Pro:** works even if the classes are not linearly separable
- Con:** the fitted model depends on very few instances (support vectors) and ignores the location of other points (as long as they are on the correct side of the decision boundary)

Soft-margin SVM

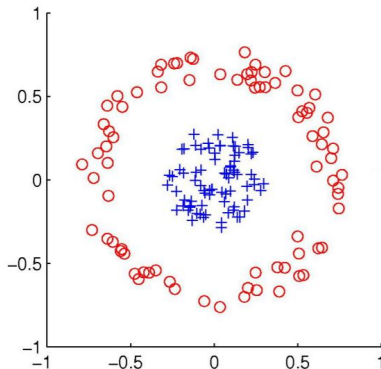
- Pro:** usually **generalizes** better than the other discussed linear methods due to margin maximization
- Pro:** can be used with kernels (discussed soon)
- Pro:** works even if the classes are not linearly separable
- Con:** the fitted model depends on very few instances (support vectors) and ignores the location of other points (as long as they are on the correct side of the decision boundary)
- Con:** Works efficiently on relatively small datasets

Motivation for Kernels

- We know that KNN results in non-linear models (decision boundary is not a straight line / hyperplane)

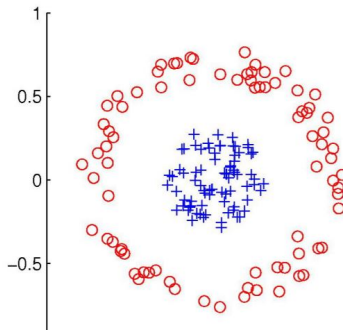
Motivation for Kernels

- We know that KNN results in non-linear models (decision boundary is not a straight line / hyperplane)
- For example, KNN works very nicely on these data:



Motivation for Kernels

- We know that KNN results in non-linear models (decision boundary is not a straight line / hyperplane)
- For example, KNN works very nicely on these data:



It turns out that linear methods can also learn this separation! How?

Non-linearity with linear models

- Linear models are linear in the features

Non-linearity with linear models

- Linear models are linear in the features
- Let us construct new features that are nonlinear in the original features

Non-linearity with linear models

- Linear models are linear in the features
- Let us construct new features that are nonlinear in the original features
- Linear models on the new features are nonlinear in the original features!

Non-linearity with linear models

- Linear models are linear in the features
- Let us construct new features that are nonlinear in the original features
- Linear models on the new features are nonlinear in the original features!
- This way we can fit non-linear models using linear model learning algorithms e.g.

Non-linearity with linear models

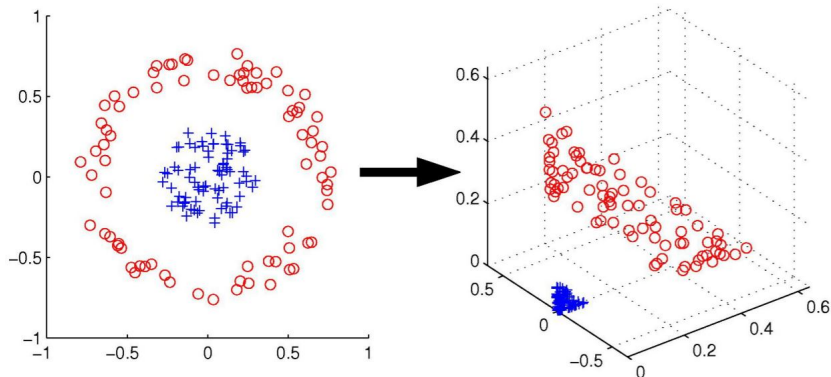
- Linear models are linear in the features
- Let us construct new features that are nonlinear in the original features
- Linear models on the new features are nonlinear in the original features!
- This way we can fit non-linear models using linear model learning algorithms e.g.
 - if the hidden dependency is $y = x^3 + 3x^2 - x + 7$
 - Then if we introduce features x, x^2, x^3 then linear methods can learn this functional dependency

Non-linearity with linear models

Let's construct new features to make data linearly separable:

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$(x_1, x_2) \rightarrow (z_1, z_2, z_3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$



Non-linearity with linear models

- Fitting:

Non-linearity with linear models

- Fitting:
 - Transform the training data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$

Non-linearity with linear models

- Fitting:
 - Transform the training data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - Learn a linear model

Non-linearity with linear models

- Fitting:
 - Transform the training data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - Learn a linear model
- Predicting:

Non-linearity with linear models

- Fitting:
 - Transform the training data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - Learn a linear model
- Predicting:
 - Transform the test data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$

Non-linearity with linear models

- Fitting:
 - Transform the training data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - Learn a linear model
- Predicting:
 - Transform the test data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - Apply the fitted linear model

Non-linearity with linear models

- Fitting:
 - Transform the training data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - Learn a linear model
- Predicting:
 - Transform the test data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - Apply the fitted linear model
- This can be very slow, if there are a lot of constructed features. Can we make it faster?

Non-linearity with linear models

- Fitting:
 - Transform the training data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - Learn a linear model
- Predicting:
 - Transform the test data through $\phi(\mathbf{x})$: $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - Apply the fitted linear model
- This can be very slow, if there are a lot of constructed features. Can we make it faster?
- Let's see how $\phi(\mathbf{x})$ is used in SVM

SVM on constructed features

- Fitting (dual form soft-margin):

$\mathbf{x}_i \rightarrow \phi(\mathbf{x}_i)$ for $i = 1, \dots, n$

$$\alpha_1^*, \dots, \alpha_n^* = \operatorname{argmax}_{\alpha_1, \dots, \alpha_n} -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j) + \sum_{i=1}^n \alpha_i$$

$$\text{subject to } 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \text{ and } \sum_{i=1}^n \alpha_i y_i = 0$$

$$\mathbf{w}^* = \sum_{i=1}^n \alpha_i^* y_i \phi(\mathbf{x}_i) \quad t^* = \mathbf{w}^* \cdot \phi(\mathbf{x}_j) - y_j = \sum_{i=1}^n \alpha_i^* y_i \phi(\mathbf{x}_i) \phi(\mathbf{x}_j) - y_j$$

SVM on constructed features

- Fitting (dual form soft-margin):

$\mathbf{x}_i \rightarrow \phi(\mathbf{x}_i)$ for $i = 1, \dots, n$

$$\alpha_1^*, \dots, \alpha_n^* = \operatorname{argmax}_{\alpha_1, \dots, \alpha_n} -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j) + \sum_{i=1}^n \alpha_i$$

$$\text{subject to } 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \text{ and } \sum_{i=1}^n \alpha_i y_i = 0$$

$$\mathbf{w}^* = \sum_{i=1}^n \alpha_i^* y_i \phi(\mathbf{x}_i) \quad t^* = \mathbf{w}^* \cdot \phi(\mathbf{x}_j) - y_j = \sum_{i=1}^n \alpha_i^* y_i \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j) - y_j$$

- Prediction:

$\mathbf{x} \rightarrow \phi(\mathbf{x})$

$$\hat{y} = \operatorname{sign}(\mathbf{w}^* \cdot \phi(\mathbf{x}) - t^*) = \operatorname{sign}\left(\sum_{i=1}^n \alpha_i^* y_i \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}) - t^*\right)$$

Everything through dot products

- In SVM fitting and prediction all uses of instances can be made through dot products

Everything through dot products

- In SVM fitting and prediction all uses of instances can be made through dot products
- Our example transformation:

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$(x_1, x_2) \rightarrow (z_1, z_2, z_3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

Everything through dot products

- In SVM fitting and prediction all uses of instances can be made through dot products
- Our example transformation:

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$(x_1, x_2) \rightarrow (z_1, z_2, z_3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

- Dot product:

$$\phi(\mathbf{x}) \cdot \phi(\mathbf{x}') = \phi(x_1, x_2) \cdot \phi(x'_1, x'_2) =$$

Everything through dot products

- In SVM fitting and prediction all uses of instances can be made through dot products
- Our example transformation:

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$(x_1, x_2) \rightarrow (z_1, z_2, z_3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

- Dot product:

$$\begin{aligned}\phi(\mathbf{x}) \cdot \phi(\mathbf{x}') &= \phi(x_1, x_2) \cdot \phi(x'_1, x'_2) = \\ &= (x_1^2, \sqrt{2}x_1x_2, x_2^2) \cdot (x_1'^2, \sqrt{2}x_1'x_2', x_2'^2) =\end{aligned}$$

Everything through dot products

- In SVM fitting and prediction all uses of instances can be made through dot products
- Our example transformation:

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$(x_1, x_2) \rightarrow (z_1, z_2, z_3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

- Dot product:

$$\begin{aligned}\phi(\mathbf{x}) \cdot \phi(\mathbf{x}') &= \phi(x_1, x_2) \cdot \phi(x'_1, x'_2) = \\ &= (x_1^2, \sqrt{2}x_1x_2, x_2^2) \cdot (x_1'^2, \sqrt{2}x_1'x_2', x_2'^2) = \\ &= x_1^2x_1'^2 + \sqrt{2}x_1x_2\sqrt{2}x_1'x_2' + x_2^2x_2'^2 =\end{aligned}$$

Everything through dot products

- In SVM fitting and prediction all uses of instances can be made through dot products
- Our example transformation:

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$(x_1, x_2) \rightarrow (z_1, z_2, z_3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

- Dot product:

$$\begin{aligned}\phi(\mathbf{x}) \cdot \phi(\mathbf{x}') &= \phi(x_1, x_2) \cdot \phi(x'_1, x'_2) = \\ &= (x_1^2, \sqrt{2}x_1x_2, x_2^2) \cdot (x_1'^2, \sqrt{2}x_1'x_2', x_2'^2) = \\ &= x_1^2x_1'^2 + \sqrt{2}x_1x_2\sqrt{2}x_1'x_2' + x_2^2x_2'^2 = \\ &= (x_1x_1')^2 + 2(x_1x_1')(x_2x_2') + (x_2x_2')^2 =\end{aligned}$$

Everything through dot products

- In SVM fitting and prediction all uses of instances can be made through dot products
- Our example transformation:

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$(x_1, x_2) \rightarrow (z_1, z_2, z_3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

- Dot product:

$$\begin{aligned}\phi(\mathbf{x}) \cdot \phi(\mathbf{x}') &= \phi(x_1, x_2) \cdot \phi(x'_1, x'_2) = \\ &= (x_1^2, \sqrt{2}x_1x_2, x_2^2) \cdot (x_1'^2, \sqrt{2}x_1'x_2', x_2'^2) = \\ &= x_1^2x_1'^2 + \sqrt{2}x_1x_2\sqrt{2}x_1'x_2' + x_2^2x_2'^2 = \\ &= (x_1x_1')^2 + 2(x_1x_1')(x_2x_2') + (x_2x_2')^2 = \\ &= (x_1x_1' + x_2x_2')^2 = (\mathbf{x} \cdot \mathbf{x}')^2\end{aligned}$$

Kernel trick

- We can now use the kernel trick!

Kernel trick

- We can now use the kernel trick!
- Instead of

Kernel trick

- We can now use the kernel trick!
- Instead of
 - transforming the instances with $\mathbf{x} \rightarrow \phi(\mathbf{x})$

- We can now use the kernel trick!
- Instead of
 - transforming the instances with $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - then fitting and predicting on the constructed higher-dimensional instances $\phi(\mathbf{x})$

- We can now use the kernel trick!
- Instead of
 - transforming the instances with $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - then fitting and predicting on the constructed higher-dimensional instances $\phi(\mathbf{x})$
- We can now do the kernel trick:

- We can now use the kernel trick!
- Instead of
 - transforming the instances with $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - then fitting and predicting on the constructed higher-dimensional instances $\phi(\mathbf{x})$
- We can now do the kernel trick:
 - Work with original instances

- We can now use the kernel trick!
- Instead of
 - transforming the instances with $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - then fitting and predicting on the constructed higher-dimensional instances $\phi(\mathbf{x})$
- We can now do the kernel trick:
 - Work with original instances
 - Use a modified method of calculating dot products $\mathbf{x} \cdot \mathbf{x}' \rightarrow \kappa(\mathbf{x}, \mathbf{x}')$

- We can now use the kernel trick!
- Instead of
 - transforming the instances with $\mathbf{x} \rightarrow \phi(\mathbf{x})$
 - then fitting and predicting on the constructed higher-dimensional instances $\phi(\mathbf{x})$
- We can now do the kernel trick:
 - Work with original instances
 - Use a modified method of calculating dot products $\mathbf{x} \cdot \mathbf{x}' \rightarrow \kappa(\mathbf{x}, \mathbf{x}')$
 - In our example the **kernel** is $\kappa(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}')^2$

- Fitting (soft-margin kernel SVM):

$$\alpha_1^*, \dots, \alpha_n^* = \operatorname{argmax}_{\alpha_1, \dots, \alpha_n} -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j) + \sum_{i=1}^n \alpha_i$$

$$\text{subject to } 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \text{ and } \sum_{i=1}^n \alpha_i y_i = 0$$

$$t^* = \sum_{i=1}^n \alpha_i^* y_i \kappa(\mathbf{x}_i, \mathbf{x}_j) - y_j,$$

where $\mathbf{x}_j$ is a support vector of class y_j .

- Fitting (soft-margin kernel SVM):

$$\alpha_1^*, \dots, \alpha_n^* = \operatorname{argmax}_{\alpha_1, \dots, \alpha_n} -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j) + \sum_{i=1}^n \alpha_i$$

$$\text{subject to } 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \text{ and } \sum_{i=1}^n \alpha_i y_i = 0$$

$$t^* = \sum_{i=1}^n \alpha_i^* y_i \kappa(\mathbf{x}_i, \mathbf{x}_j) - y_j,$$

where $\mathbf{x}_j$ is a support vector of class y_j .

- Prediction:

$$\hat{y} = \operatorname{sign} \left(\sum_{i=1}^n \alpha_i^* y_i \kappa(\mathbf{x}_i, \mathbf{x}) - t^* \right)$$

When does the kernel trick work?

- Can we choose any function for $\kappa(\mathbf{x}, \mathbf{x}')$?

When does the kernel trick work?

- Can we choose any function for $\kappa(\mathbf{x}, \mathbf{x}')$?
- No! We can only choose kernels for which there exists some transformation ϕ such that:

$$\kappa(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$$

Otherwise the mathematics of SVM breaks down

When does the kernel trick work?

- Can we choose any function for $\kappa(\mathbf{x}, \mathbf{x}')$?
- No! We can only choose kernels for which there exists some transformation ϕ such that:

$$\kappa(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$$

Otherwise the mathematics of SVM breaks down

- Mercer's theorem:
Such transformation ϕ exists if and only if $\kappa(\mathbf{x}, \mathbf{x}')$ is a positive semi-definite function

Positive semi-definiteness

Definition

Let $\kappa : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$, then function $\kappa(\mathbf{x}, \mathbf{x}')$ is positive semi-definite, if for any $\mathbf{x}_1, \dots, \mathbf{x}_n$ the following matrix (**kernel matrix**) is positive semi-definite:

$$\begin{pmatrix} \kappa(\mathbf{x}_1, \mathbf{x}_1) & \kappa(\mathbf{x}_1, \mathbf{x}_2) & \dots & \kappa(\mathbf{x}_1, \mathbf{x}_n) \\ \kappa(\mathbf{x}_2, \mathbf{x}_1) & \kappa(\mathbf{x}_2, \mathbf{x}_2) & \dots & \kappa(\mathbf{x}_2, \mathbf{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ \kappa(\mathbf{x}_n, \mathbf{x}_1) & \kappa(\mathbf{x}_n, \mathbf{x}_2) & \dots & \kappa(\mathbf{x}_n, \mathbf{x}_n) \end{pmatrix}$$

Definition

A matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$ is positive semi-definite, if for any vector $\mathbf{x} \in \mathbb{R}^n$

$$\mathbf{x}^T \mathbf{M} \mathbf{x} \geq 0$$

Checking these criteria might be non-trivial, often new kernels are constructed by combining existing ones

Well-known kernels

- Linear kernel (original space unchanged):

$$\kappa(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$$

Well-known kernels

- Linear kernel (original space unchanged):

$$\kappa(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$$

- Polynomial kernel (for any $\gamma, r \in \mathbb{R}$ and $d \in \mathbb{N}$):

$$\kappa(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x} \cdot \mathbf{x}' + r)^d$$

- Linear kernel (original space unchanged):

$$\kappa(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$$

- Polynomial kernel (for any $\gamma, r \in \mathbb{R}$ and $d \in \mathbb{N}$):

$$\kappa(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x} \cdot \mathbf{x}' + r)^d$$

- Gaussian kernel (also known as Radial Basis Function (RBF)) for any $\sigma > 0$ ($\gamma > 0$):

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma}\right) = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$$

Polynomial kernel

- For any $\gamma, r \in \mathbb{R}$ and $d \in \mathbb{N}$:

$$\kappa(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x} \cdot \mathbf{x}' + r)^d$$

Polynomial kernel

- For any $\gamma, r \in \mathbb{R}$ and $d \in \mathbb{N}$:

$$\kappa(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x} \cdot \mathbf{x}' + r)^d$$

- For example, take $\gamma = r = 1$ and $d = 2$ with 2 features:

$$\kappa(\mathbf{x}, \mathbf{x}') = (x_1 x'_1 + x_2 x'_2 + 1)^2 =$$

Polynomial kernel

- For any $\gamma, r \in \mathbb{R}$ and $d \in \mathbb{N}$:

$$\kappa(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x} \cdot \mathbf{x}' + r)^d$$

- For example, take $\gamma = r = 1$ and $d = 2$ with 2 features:

$$\begin{aligned}\kappa(\mathbf{x}, \mathbf{x}') &= (x_1 x'_1 + x_2 x'_2 + 1)^2 = \\ &= x_1^2 x_1'^2 + x_2^2 x_2'^2 + 1 \cdot 1 + \sqrt{2} x_1 x_2 \sqrt{2} x'_1 x'_2 + \sqrt{2} x_1 \sqrt{2} x'_1 + \sqrt{2} x_2 \sqrt{2} x'_2 =\end{aligned}$$

Polynomial kernel

- For any $\gamma, r \in \mathbb{R}$ and $d \in \mathbb{N}$:

$$\kappa(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x} \cdot \mathbf{x}' + r)^d$$

- For example, take $\gamma = r = 1$ and $d = 2$ with 2 features:

$$\begin{aligned}\kappa(\mathbf{x}, \mathbf{x}') &= (x_1 x'_1 + x_2 x'_2 + 1)^2 = \\ &= x_1^2 x_1'^2 + x_2^2 x_2'^2 + 1 \cdot 1 + \sqrt{2} x_1 x_2 \sqrt{2} x'_1 x'_2 + \sqrt{2} x_1 \sqrt{2} x'_1 + \sqrt{2} x_2 \sqrt{2} x'_2 = \\ &= (x_1^2, x_2^2, 1, \sqrt{2} x_1 x_2, \sqrt{2} x_1, \sqrt{2} x_2) \cdot (x_1'^2, x_2'^2, 1, \sqrt{2} x'_1 x'_2, \sqrt{2} x'_1, \sqrt{2} x'_2) =\end{aligned}$$

Polynomial kernel

- For any $\gamma, r \in \mathbb{R}$ and $d \in \mathbb{N}$:

$$\kappa(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x} \cdot \mathbf{x}' + r)^d$$

- For example, take $\gamma = r = 1$ and $d = 2$ with 2 features:

$$\begin{aligned}\kappa(\mathbf{x}, \mathbf{x}') &= (x_1 x'_1 + x_2 x'_2 + 1)^2 = \\&= x_1^2 x_1'^2 + x_2^2 x_2'^2 + 1 \cdot 1 + \sqrt{2} x_1 x_2 \sqrt{2} x'_1 x'_2 + \sqrt{2} x_1 \sqrt{2} x'_1 + \sqrt{2} x_2 \sqrt{2} x'_2 = \\&= (x_1^2, x_2^2, 1, \sqrt{2} x_1 x_2, \sqrt{2} x_1, \sqrt{2} x_2) \cdot (x_1'^2, x_2'^2, 1, \sqrt{2} x'_1 x'_2, \sqrt{2} x'_1, \sqrt{2} x'_2) = \\&= \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')\end{aligned}$$

Polynomial kernel

- For any $\gamma, r \in \mathbb{R}$ and $d \in \mathbb{N}$:

$$\kappa(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x} \cdot \mathbf{x}' + r)^d$$

- For example, take $\gamma = r = 1$ and $d = 2$ with 2 features:

$$\begin{aligned}\kappa(\mathbf{x}, \mathbf{x}') &= (x_1 x'_1 + x_2 x'_2 + 1)^2 = \\&= x_1^2 x_1'^2 + x_2^2 x_2'^2 + 1 \cdot 1 + \sqrt{2} x_1 x_2 \sqrt{2} x'_1 x'_2 + \sqrt{2} x_1 \sqrt{2} x'_1 + \sqrt{2} x_2 \sqrt{2} x'_2 = \\&= (x_1^2, x_2^2, 1, \sqrt{2} x_1 x_2, \sqrt{2} x_1, \sqrt{2} x_2) \cdot (x_1'^2, x_2'^2, 1, \sqrt{2} x'_1 x'_2, \sqrt{2} x'_1, \sqrt{2} x'_2) = \\&= \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')\end{aligned}$$

- Constructs all polynomials of features x_1, x_2 up to $d = 2$ degree

In case of Gaussian kernel the constructed feature space is infinite-dimensional !!! E.g. for $\sigma = 1$ and $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^k$

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp \left(- \frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2} \right) =$$

In case of Gaussian kernel the constructed feature space is infinite-dimensional !!! E.g. for $\sigma = 1$ and $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^k$

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2}\right) = e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} e^{\mathbf{x} \cdot \mathbf{x}'} =$$

Gaussian kernel

In case of Gaussian kernel the constructed feature space is infinite-dimensional !!! E.g. for $\sigma = 1$ and $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^k$

$$\begin{aligned}\kappa(\mathbf{x}, \mathbf{x}') &= \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2}\right) = e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} e^{\mathbf{x} \cdot \mathbf{x}'} = \\ &= e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} \sum_{j=0}^{\infty} \frac{(\mathbf{x} \cdot \mathbf{x}')^j}{j!} =\end{aligned}$$

Gaussian kernel

In case of Gaussian kernel the constructed feature space is infinite-dimensional !!! E.g. for $\sigma = 1$ and $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^k$

$$\begin{aligned}\kappa(\mathbf{x}, \mathbf{x}') &= \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2}\right) = e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} e^{\mathbf{x} \cdot \mathbf{x}'} = \\ &= e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} \sum_{j=0}^{\infty} \frac{(\mathbf{x} \cdot \mathbf{x}')^j}{j!} =\end{aligned}$$

using this formula $(x_1 + \dots + x_k)^d = \sum_{n_i \geq 0, \sum_i n_i = d} \frac{d!}{n_1! \dots n_k!} x_1^{n_1} \dots x_k^{n_k}$

Gaussian kernel

In case of Gaussian kernel the constructed feature space is infinite-dimensional !!! E.g. for $\sigma = 1$ and $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^k$

$$\begin{aligned}\kappa(\mathbf{x}, \mathbf{x}') &= \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2}\right) = e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} e^{\mathbf{x} \cdot \mathbf{x}'} = \\ &= e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} \sum_{j=0}^{\infty} \frac{(\mathbf{x} \cdot \mathbf{x}')^j}{j!} =\end{aligned}$$

using this formula $(x_1 + \dots + x_k)^d = \sum_{n_i \geq 0, \sum_i n_i = d} \frac{d!}{n_1! \dots n_k!} x_1^{n_1} \dots x_k^{n_k}$

$$= e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} \sum_{j=0}^{\infty} \frac{1}{j!} \sum_{\sum_i n_i = j} \frac{j! (x_1 x'_1)^{n_1} \dots (x_k x'_k)^{n_k}}{n_1! \dots n_k!} =$$

Gaussian kernel

In case of Gaussian kernel the constructed feature space is infinite-dimensional !!! E.g. for $\sigma = 1$ and $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^k$

$$\begin{aligned}\kappa(\mathbf{x}, \mathbf{x}') &= \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2}\right) = e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} e^{\mathbf{x} \cdot \mathbf{x}'} = \\ &= e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} \sum_{j=0}^{\infty} \frac{(\mathbf{x} \cdot \mathbf{x}')^j}{j!} =\end{aligned}$$

using this formula $(x_1 + \dots + x_k)^d = \sum_{n_i \geq 0, \sum_i n_i = d} \frac{d!}{n_1! \dots n_k!} x_1^{n_1} \dots x_k^{n_k}$

$$\begin{aligned}&= e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} \sum_{j=0}^{\infty} \frac{1}{j!} \sum_{\sum_i n_i = j} \frac{j! (x_1 x'_1)^{n_1} \dots (x_k x'_k)^{n_k}}{n_1! \dots n_k!} = \\ &= \sum_{j=0}^{\infty} \sum_{\sum_i n_i = j} e^{-\frac{\|\mathbf{x}\|^2}{2}} \frac{x_1^{n_1} \dots x_k^{n_k}}{\sqrt{n_1! \dots n_k!}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} \frac{x_1'^{n_1} \dots x_k'^{n_k}}{\sqrt{n_1! \dots n_k!}}\end{aligned}$$

Gaussian kernel

In case of Gaussian kernel the constructed feature space is infinite-dimensional !!! E.g. for $\sigma = 1$ and $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^k$

$$\begin{aligned}\kappa(\mathbf{x}, \mathbf{x}') &= \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2}\right) = e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} e^{\mathbf{x} \cdot \mathbf{x}'} = \\ &= e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} \sum_{j=0}^{\infty} \frac{(\mathbf{x} \cdot \mathbf{x}')^j}{j!} =\end{aligned}$$

using this formula $(x_1 + \dots + x_k)^d = \sum_{n_i \geq 0, \sum_i n_i = d} \frac{d!}{n_1! \dots n_k!} x_1^{n_1} \dots x_k^{n_k}$

$$\begin{aligned}&= e^{-\frac{\|\mathbf{x}\|^2}{2}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} \sum_{j=0}^{\infty} \frac{1}{j!} \sum_{\sum_i n_i = j} \frac{j! (x_1 x'_1)^{n_1} \dots (x_k x'_k)^{n_k}}{n_1! \dots n_k!} = \\ &= \sum_{j=0}^{\infty} \sum_{\sum_i n_i = j} e^{-\frac{\|\mathbf{x}\|^2}{2}} \frac{x_1^{n_1} \dots x_k^{n_k}}{\sqrt{n_1! \dots n_k!}} e^{-\frac{\|\mathbf{x}'\|^2}{2}} \frac{x'_1{}^{n_1} \dots x'_k{}^{n_k}}{\sqrt{n_1! \dots n_k!}}\end{aligned}$$

Here the feature map is the following: $\phi(\mathbf{x}) = \left(e^{-\frac{\|\mathbf{x}\|^2}{2}} \frac{x_1^{n_1} \dots x_k^{n_k}}{\sqrt{n_1! \dots n_k!}} \right)_{\sum_{i=1}^k n_i = j}$

Construction of kernels

If κ_1, κ_2 are kernels, $a > 0$ and $f : \mathbb{X} \rightarrow \mathbb{R}$ then the following $\kappa(\mathbf{x}, \mathbf{x}')$ are kernels as well:

$$\kappa(\mathbf{x}, \mathbf{x}') = a\kappa_1(\mathbf{x}, \mathbf{x}')$$

$$\kappa(\mathbf{x}, \mathbf{x}') = \kappa_1(\mathbf{x}, \mathbf{x}')\kappa_2(\mathbf{x}, \mathbf{x}')$$

$$\kappa(\mathbf{x}, \mathbf{x}') = \kappa_1(\mathbf{x}, \mathbf{x}') + \kappa_2(\mathbf{x}, \mathbf{x}')$$

$$\kappa(\mathbf{x}, \mathbf{x}') = f(\mathbf{x})f(\mathbf{x}')$$

What have we learned today?

- ✓ Convex Optimization
- ✓ Support Vector Machine (SVM)
- ✓ Hard-margin SVM
- ✓ Soft-margin SVM
- ✓ Kernel Methods